Formal Languages and Compilers

(Linguaggi Formali e Compilatori)

prof. Luca Breveglieri

(prof. S. Crespi Reghizzi, prof. A. Morzenti)

Written exam - 25 September 2013 - Part I: Theory

WARNING - THE SYNTAX ANALYSIS EXERCISES MUST BE SOLVED BY THE ELR / ELL THEORY

NAME:

SURNAME:

ID: SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam consists of two parts:
  - I (80%) Theory:
    1. regular expressions and finite automata
    2. free grammars and pushdown automata
    3. syntax analysis and parsing
    4. translation and semantic analysis
  - II (20%) Practice on Flex and Bison
- To pass the exam, the candidate must succeed in both parts (I and II), in one call or more calls separately, but within one year.
- To pass part I (theory) one must answer the mandatory (not optional) questions. Notice the full grade is achieved by answering the optional questions.
- The exam is open book (texts and personal notes are admitted).
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets nor replace the existing ones.
- Time: Part I (theory): 2h.15m - Part II (practice): 60m

1 Regular Expressions and Finite Automata 20%

1. Consider the following grammar G, without auto-inclusive (self-embedding) derivations (axiom A):

$$G \begin{cases} A & \rightarrow & a\ B\ b\ A \ \mid\ \varepsilon \\ B & \rightarrow & c\ d\ B \ \mid\ \varepsilon \end{cases}$$

Answer the following questions:

(a) Write a regular expression R for language $L(G)$, preferably by considering the rules of grammar G as set equations associated with the languages of the non-terminals, and by applying substitution.

(b) (optional) Say if the language $L(G)$ is local and explain why.

(c) By using the simplest methodology suited for the purpose, design a finite state deterministic automaton A that recognizes the language $L(G)$.

(d) If necessary, minimize the automaton A obtained at point (c).

---

2 Free Grammars and Pushdown Automata 20%

1. Consider the following grammar G (axiom S), of type *BNF* and non-ambiguous, over the terminal alphabet $\Sigma = \{\ a,\ b,\ c\ \}$:

   $$G\colon\ S \rightarrow a\ S\ a\ \mid\ b\ S\ b\ \mid\ c$$

   Grammar G generates the language of the palindromes of letters a and b, with letter c in the centre.

   Answer the following questions:

   (a) Write a grammar G', of type *BNF* and non-ambiguous, that generates the difference language $L\,(G) - \{\ a\,c\,a\ \}$, i.e., the palindrome language without the one palindrome $a\,c\,a$.

   (b) Write a grammarG'', of type *BNF* and non-ambiguous, that generates the difference language $L\,(G) - \{\ a^n\ c\ a^n \mid \quad n \geq 1\ \}$, i.e., the palindrome language without the infinitely many palindromes $a^n\,c\,a^n$ for every $n \geq 1$.

   (c) (optional) Write a grammar G''', of type *BNF* and non-ambiguous, that generates the difference language $L\,(G) - \Sigma^*\,b\,a\,\Sigma^*$, i.e., the palindrome languages without the palindromes where a letter a immediately follows a letter b.

---

2. In the *PL/I* language there are various types of iterative constructs, illustrated below by means of a few examples. These examples also contain instructions that are different from the iterative ones, as well as various types of expression; all these instructions and expressions are not part of the current exercise, which focuses on `DO`.

simple DO statement (the statements in the DO-body are executed one time only) Ex.:

```
DO;
   PUT LIST ('More data');
   GET LIST (VALUE);
   C = C + VALUE;
END;
```

DO REPEAT statement Example:

```
DECLARE (HEAD, P, NEXT) POINTER;
...
DO P = HEAD REPEAT (P->NODE.NEXT)
   WHILE (P^ = NULL);
   ...
END;
```

DO WHILE and DO UNTIL statements Examples:

```
DO WHILE (EOF = 0);
   READ FILE(F) INTO(REC);
END;

DO UNTIL (B = 1);
   ARRAY(B) = B;
   B = B - 1;
END;

DO WHILE (EOF = 0)
   UNTIL (ALPHA = 'end');
   READ FILE(F) INTO(REC);
END;
```

iterative DO statement Ex.s:

```
DO K = 1 TO 10;
   ...
END;
DO K = 10 TO 1 BY -1;
   ...
END;
DO K = 1 TO 10 WHILE (B < 0);
   ...
END;
DO A(J) = B + 1 TO N BY -M
   WHILE (T);
   ...
END;
```

DO list statement In this form none of the TO, BY or REPEAT options are used, but multiple instances of the start expression are specified. The body is executed once for each specified start value. Example:

```
DO name 'Bob', 'Ann', 'Fred', 'Joan';
   READ FILE(F) KEY(name) INTO(REC);
   CALL SEND_REPT;
END;
```

complex iterative DO statement It can include multiple instances of the iteration-spec. The DO list statement above is a simple example. Example:

```
DO J = 1 TO 9
   WHILE (B(J)^ -= 'xx'),
   J = 16 TO 20 BY 2;
   READ FILE(F) INTO(REC);
   B(J) = SUBSTR(REC,6,2);
END;
```

Here are a few details: in the control clauses of the `DO` construct, the *expressions* and the *variables* must be symbolized by means of the the terminals `exp` and `var`, respectively; the body of the `DO` construct contains one or more instructions, and may contain even one or more inner `DO` constructs; every other instruction must be generically symbolized by means of the terminal `stat`; and the variable declarations must be ignored.

Write a grammar G, of type *EBNF* and not ambiguous, that generates such `DO` constructs. It is preferable that the structure of grammar G reflects the previous classification of `DO` constructs.

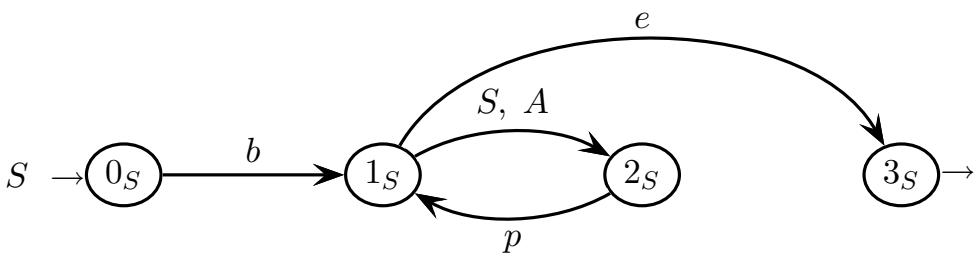
---

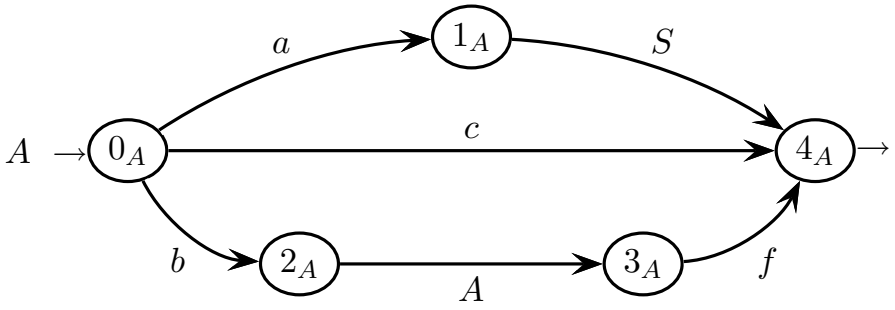
3 Syntax Analysis and Parsing 20%

1. Consider the following grammar G (axiom S), of type *EBNF*:

$$G \begin{cases} S & \rightarrow & b\,\big(\,(\,S \;\mid\; A\,)\,p\,\big)^*\,e \\ A & \rightarrow & a\,S \;\mid\; c \;\mid\; b\,A\,f \end{cases}$$

Grammar G is represented as a network of recursive machines, as follows:





Answer the following questions:

(a) Draw a part of the pilot sufficient to prove that grammar G is not of type $ELL\,(1)$.

(b) Consider the string $b\,b\,c\,f\,p\,e \in L\,(G)$ and draw its syntax tree.

(c) Draw a part of the pilot sufficient for the bottom-up syntactic analysis of the string $b\,b\,c\,f\,p\,e \in L\,(G)$, and verify that such a part of the pilot satisfies the condition $ELR\,(1)$.

(d) (optional) Outline the simulation trace of the bottom-up analysis of such a string, and show the shift and reduce operations executed by the analyzer, as well as the syntax tree construction.

CAUTION: read the warning on the text cover page.

---

4 Translation and Semantic Analysis 20%

1. Consider the boolean expressions (in the infix form) with binary operators *or* "$\vee$" and *and* "$\wedge$", direct and negated variables respectively represented by a and b, and parentheses "(" and ")" at any nesting depth. Here is the grammar G of these expressions (axiom E), of type *BNF* and non-ambiguous:

$$G \begin{cases} E & \rightarrow & E \vee T \mid T \\ T & \rightarrow & T \wedge F \mid F \\ F & \rightarrow & a \mid b \mid (E) \end{cases}$$

The operator precedences are: *and* precedes *or*. Here are three sample expressions:

$$a \vee b \wedge a \qquad\qquad a \vee (a \wedge a) \qquad\qquad a \wedge (a \vee b)$$

The parentheses in the second expression are unnecessary, but are permitted.

The dual form of a boolean expression is defined by swapping *or* and *and*, and by swapping directed and negated variables. The parentheses present in the source expression are preserved in the dual form. For instance:

$$b \vee a \wedge (a \vee b) \vee a \Rightarrow \underbrace{a \wedge \big(b \vee (b \wedge a) \big) \wedge b}_{\text{dual form}}$$

Notice that the precedences of the operators contained in the source expression have to be preserved. Therefore it may be sometimes necessary to add parentheses to the dual form (see the example above).

Answer the following questions:

(a) Write a translation grammar G_τ (or a syntactic translation scheme), of type (*BNF*) and non-ambiguous, that computes the dual form of the boolean expressions, and draw the translation tree of the example above.

(b) (optional) Consider boolean expressions defined as above, but without parentheses. Write a regular translation expression R_τ, not ambiguous, that computes the dual form of the boolean expressions without parentheses. It may happen that the translated expression contains parentheses, to keep the precedences.

---

2. One wishes one searched how many times a given identifier (*query*) occurs in a set of records and atomic objects. A record may contain nested records and atomic fields. The following syntactic support describes such sets:

$$L \rightarrow \text{id}: \text{record } L \text{ end } L \qquad \text{- - "id" has the right attribute } n$$

$$L \rightarrow \text{id}: \text{atomic } L \qquad \text{- - idem}$$

$$L \rightarrow \text{id}: \text{atomic} \qquad \text{- - idem}$$

The identifier "id" is associated with a right attribute n initialized with the name of the record or atomic object.

The name (which is a string) to be searched is stored in the right attributed q of the terminal "query" in the following axiomatic rule:

$$S \rightarrow L \text{ query} \qquad \text{- - "query" has the right attribute } q$$

For instance, in the following records:

ALPHA: record *GAMMA*: atomic end *BETA*: record *ALPHA*: atomic end

the name *ALPHA* occurs twice, while each name *GAMMA* and *BETA* occurs only once.

Answer the following questions:

(a) Write an attribute grammar that computes, in the attribute *tot*, the number of occurrences of the name q. Write the semantic rules in the frame on the next pages. If more attributes are used, list them in the attribute table below.
Attribute table (predefined and to be possibly added) for question (a):

type	*name*	*association*	*domain*	*meaning*
right	q	query L	string	name to be searched
right	n	id	string	name of record or field
left	*tot*	L S	integer	number of occurrences

(b) (optional) Write at least partially the semantic procedures that compute the attributes in one sweep. If it were impossible to do one-sweep, explain why.

#	*syntax*	*semantic* - question (a) - to be completed
1:	$S_0 \rightarrow L_1$ query$_2$	
2:	$L_0 \rightarrow$ id$_1$: record L_2 end L_3	
3:	$L_0 \rightarrow$ id$_1$: atomic L_2	
4:	$L_0 \rightarrow$ id$_1$: atomic	